

C vs C++ — Headers and Features Differences

1. Header Files in C but NOT in C++

`<tgmath.h>` — Type-Generic Math (C99)

C uses macros to auto-pick the right function. C++ doesn't need this — overloading does it.

```
// C
#include <tgmath.h>
sqrt(2.0f);    // macro picks sqrtf
sqrt(2.0);     // macro picks sqrt
```

```
// C++ - same call, overloading handles it
#include <cmath>
std::sqrt(2.0f); // auto picks float version
std::sqrt(2.0);  // auto picks double version
```

`<stdatomic.h>` — C11 Atomics

C++ has `<atomic>` instead (different API).

```
// C
#include <stdatomic.h>
atomic_int counter = 0;
atomic_fetch_add(&counter, 1);
```

```
// C++ - different API
#include <atomic>
std::atomic<int> counter = 0;
counter++;
```

`<threads.h>` — C11 Threads

C++ has `<thread>`, `<mutex>`, `<condition_variable>` (different API).

```
// C
#include <threads.h>
thrd_t t;
```

```
thrd_create(&t, myFunc, NULL);
thrd_join(t, NULL);
```

```
// C++
#include <thread>
std::thread t(myFunc);
t.join();
```

<stdnoreturn.h> — C11

C++ uses `[[noreturn]]` attribute (no header needed).

```
// C
#include <stdnoreturn.h>
noreturn void crash() { exit(1); }
```

```
// C++
[[noreturn]] void crash() { exit(1); }
```

<stdalign.h> — C11

C++ has `alignas/alignof` as built-in keywords (no header needed).

```
// C
#include <stdalign.h>
alignas(16) int data[4];
```

```
// C++ - just works without any header
alignas(16) int data[4];
```

<complex.h> — C99 (Different from C++ <complex>)

C uses `_Complex` keyword. C++ uses template class — completely different.

```
// C
#include <complex.h>
double _Complex z = 3.0 + 4.0 * I;
double mag = cabs(z);
```

```
// C++
#include <complex>
std::complex<double> z(3.0, 4.0);
double mag = std::abs(z);
```

<uchar.h> — C11 Unicode Utilities

Has partial counterpart in C++.

2. C Keywords / Features NOT in Standard C++

restrict Keyword

C99 optimization hint — tells compiler the pointer is the only access path to that memory.

```
// C
void add(int* restrict a, int* restrict b, int* restrict result, int n) {
    for (int i = 0; i < n; i++) result[i] = a[i] + b[i];
}
```

```
// C++ - no restrict in standard
// MSVC and GCC offer __restrict as non-standard extension
void add(int* __restrict a, int* __restrict b);
```

_Generic — Type-Generic Expressions (C11)

Compile-time type switching. C++ uses templates/overloading instead.

```
// C
#define printType(x) _Generic((x), \
    int: "integer", \
    double: "double", \
    char*: "string", \
    default: "unknown")

printf("%s\n", printType(42)); // integer
```

```
// C++ uses templates/overloading instead
template<typename T>
const char* printType(T) { return "unknown"; }
```

```
template<>
const char* printType(int) { return "integer"; }
```

Variable Length Arrays — VLA (C99)

Array size can be a runtime variable.

```
// C99 - OK
void makeArray(int n) {
    int arr[n];    // size known at runtime
}
```

```
// C++ - NOT standard
void makeArray(int n) {
    int arr[n];           // not standard C++ (some compilers allow)
    int* arr = new int[n]; // heap (manual)
    std::vector<int> arr(n); // BEST - automatic management
}
```

Designated Initializers (C99 full version)

```
// C99 - any order, any field, supports arrays
struct Point { int x; int y; int z; };
struct Point p = { .z = 30, .x = 10 };
int arr[10] = { [0] = 100, [5] = 500, [9] = 900 };
```

```
// C++20 - limited version: in-order only, no arrays
Point p = { .x = 10, .z = 30 };    // C++20, must be in order
int arr[10] = { [5] = 500 };      // STILL not allowed in C++
```

Compound Literals (C99)

```
// C
void printPoint(struct Point p);
printPoint((struct Point){10, 20}); // temporary struct inline

int* p = (int[]){1, 2, 3};          // temporary array
```

```
// C++ has constructors instead
```

```
printPoint(Point{10, 20});           // C++ brace initialization
```

`_Complex` and `_Imaginary` Types

```
// C - built-in language keyword
double _Complex z = 3.0 + 4.0 * I;
```

```
// C++ - template class only, NO _Complex keyword
std::complex<double> z(3.0, 4.0);
```

`_Bool` Type

```
// C99
_Bool flag = 1;
#include <stdbool.h>    // adds 'bool' macro
bool flag2 = true;
```

```
// C++ - bool is a built-in keyword
bool flag = true;
```

3. Functions in C but NOT in C++

Most standard `<math.h>` functions exist in both — `<cmath>` is a superset of `<math.h>`.

Non-standard math extensions (GNU/POSIX only) — `math.h` only

```
// GNU/POSIX extensions - NOT in <cmath>
void sincos(double x, double* s, double* c); // both at once
double exp10(double x);                      // 10^x
double pow10(double x);                      // same as exp10

// POSIX Bessel functions
double j0(double x), j1(double x);           // 1st kind
double y0(double x), y1(double x);           // 2nd kind
double jn(int n, double x), yn(int n, double x);

// Deprecated old functions
double finite(double x); // use isfinite()
double drem(double x, double y); // use remainder()
double scalb(double x, double n); // use scalbn()
```

Removed/deprecated in C++

- `gets()` - removed (was a security disaster)

4. Major Differences in Approach

Feature	C Approach	C++ Approach
Type-generic math	<code><tgmath.h></code> macros	Function overloading
Atomics	<code><stdatomic.h></code>	<code><atomic></code> template class
Threads	<code><threads.h></code>	<code><thread></code> , <code><mutex></code>
Type select	<code>_Generic</code> macro	Templates + overloading
Complex numbers	<code>_Complex</code> keyword	<code>std::complex<T></code> template
Dynamic arrays	VLA	<code>std::vector<T></code>
Memory management	<code>malloc/free</code>	<code>new/delete</code> , smart pointers
Strings	<code>char*</code> + <code><string.h></code>	<code>std::string</code>
I/O	<code>printf/scanf</code>	<code>std::cout</code> / <code>std::cin</code>
Boolean	<code>_Bool</code> / <code>bool</code> macro	<code>bool</code> keyword
Inline	<code>static inline</code>	<code>inline</code>
No return	<code>_Noreturn</code> / macro	<code>[[noreturn]]</code> attribute

5. Quick Reference — Which Headers in Which Language

C-only headers

<code><tgmath.h></code>	Type-generic math
<code><stdatomic.h></code>	C11 atomics
<code><threads.h></code>	C11 threads
<code><stdnoreturn.h></code>	C11 noreturn macro
<code><stdalign.h></code>	C11 alignment macros
<code><complex.h></code>	C99 complex numbers (with <code>_Complex</code>)
<code><uchar.h></code>	C11 unicode utilities

C++-only headers (no C equivalent)

<code><iostream></code>	C++ streams
<code><string></code>	<code>std::string</code>
<code><vector></code>	<code>std::vector</code>
<code><algorithm></code>	sort, find, etc.
<code><memory></code>	<code>unique_ptr</code> , <code>shared_ptr</code>

```

<functional>      std::function, lambdas helpers
<thread>          C++11 threads (different from C's)
<atomic>          C++11 atomics (different from C's)
<typeinfo>        RTTI
<type_traits>     SFINAE / metaprogramming
<chrono>          time utilities
<filesystem>      C++17 filesystem

```

Both (C versions usable in C++ via `<cname>`)

```

<stdio.h>   = <cstdio>
<stdlib.h>  = <cstdlib>
<string.h>  = <cstring>
<math.h>    = <cmath>
<time.h>    = <ctime>
<ctype.h>   = <cctype>
<assert.h>  = <cassert>

```

In C++, prefer `<cname>` form — it puts symbols in `std::` namespace.

6. Summary

C++ removes some C features because it provides BETTER alternatives:

C feature	C++ alternative
<code>_Generic</code>	templates + overloading
<code><tgmath.h></code>	function overloading in <code><cmath></code>
VLA	<code>std::vector<T></code>
<code><stdatomic.h></code>	<code>std::atomic<T></code>
<code><threads.h></code>	<code>std::thread</code> , <code>std::mutex</code>
<code>malloc/free</code>	<code>new/delete</code> , <code>unique_ptr</code> , <code>shared_ptr</code>
<code>_Complex</code>	<code>std::complex<T></code>
<code>restrict</code>	(no equivalent – extension only)
<code>_Noreturn</code>	<code>[[noreturn]]</code> attribute
designated init (array)	(C++20 partial only)

C++ is not "missing" these - it solved the same problems with type-safe, object-oriented mechanisms.